

Data Science & AI Interview Questions

50 Questions with Answers | Topics: ML, LLM, AI Systems, Statistics

1 Detecting Firearm Sales

Q You work at a marketplace platform. How would you build a model to detect potentially illegal firearm sales from listing text and user behavior data?

A Frame as a binary classification problem (flagged / not flagged). Feature engineering: TF-IDF or embedding of listing text for keywords (caliber, suppressor, no-serial), seller history, buyer-seller message patterns, price anomalies. Models: gradient boosted trees (XGBoost/LightGBM) for structured features + a text classifier (fine-tuned BERT) combined via a stacking ensemble. Given class imbalance, use SMOTE or class weights and optimize for recall (minimize false negatives). Add a human-review layer for borderline cases. Monitor for distribution shift as sellers adapt language.

2 Rejection Reason

Q A credit model rejects applicants. How would you build a system that explains the top reasons for each individual rejection?

A Use SHAP (SHapley Additive exPlanations) values on the trained model. For each rejected applicant, compute per-feature SHAP values and surface the top 3 negative contributors (e.g., high DTI ratio, short credit history, recent delinquency). Translate feature names into plain-language templates: 'Your debt-to-income ratio of X% exceeds our threshold.' Ensure explanations are consistent with adverse action notice requirements (ECOA/FCRA). Validate that explanations are stable and not contradictory across similar applicants.

3 Bank Fraud Model

Q You are a data scientist at a bank. How would you design an end-to-end fraud detection model for credit card transactions?

A Data: transaction amount, merchant category, location, time, device fingerprint, velocity features (transactions in last 1h/24h). Handle severe class imbalance (~0.1% fraud) with under-sampling majority or cost-sensitive learning. Model: LightGBM for speed + recall; optionally an autoencoder for anomaly detection on unlabeled data. Evaluation: precision-recall AUC, not ROC-AUC (imbalanced). Deploy as a real-time scoring API (<100ms latency). Add a feedback loop: disputed transactions retrain the model weekly. Monitor for concept drift as fraud patterns evolve.

4 Missing Housing Data

Q Your housing price dataset has 20% missing values across several features. How do you handle this before modeling?

A First, diagnose missingness type — MCAR, MAR, or MNAR — using Little's test and missingness correlation analysis. For MCAR/MAR: use iterative imputation (IterativeImputer/MICE) or KNN imputation for numerical features; mode or a dedicated 'Unknown' category for categoricals. For MNAR (e.g., income not reported by high earners): add a binary indicator feature 'was_missing' alongside imputed value to let the model learn the pattern. Avoid dropping rows unless missingness is >50% per row. Always impute on train set only, then apply the same transform to validation/test.

5 Same Algorithm Different Success

Q Two teams use the same ML algorithm on the same dataset but get very different results. What could explain this?

A Key sources of divergence: (1) Data splits — different train/val/test splits or leakage in one team's pipeline. (2) Preprocessing — different scaling, encoding, or imputation strategies. (3) Hyperparameters — different defaults or tuning budgets. (4) Feature sets — one team included features the other dropped. (5) Random seeds — stochastic algorithms vary without fixed seeds. (6) Evaluation metric — teams may optimize or report different metrics. Debugging: share reproducible pipelines, fix random seeds, align preprocessing steps, and compare feature importance rankings.

6 Booking Regression

Q You want to predict the number of hotel bookings for the next 30 days. How would you approach this regression problem?

A This is a time-series regression. Features: day of week, month, holidays, lead time, historical booking rates (lag features: bookings t-7, t-14, t-28), rolling means, promotional flags, competitor pricing. Models: start with a baseline (same day last year), then try LightGBM with lag features, or Prophet for interpretable seasonality decomposition, or LSTM for capturing long-range dependencies. Evaluate with MAE and MAPE on a rolling-window backtest. Consider prediction intervals for capacity planning. Retrain weekly with fresh data.

7 Encoding Categorical Features

Q What are the main strategies for encoding categorical features, and when would you use each?

A One-hot encoding: for nominal low-cardinality features (<10 categories) with tree or linear models. Label encoding: for ordinal features (Low/Med/High) where order is meaningful. Target encoding: for high-cardinality features (e.g., zip code) — replace each category with the mean target value; use cross-validation to prevent leakage. Frequency/count encoding: useful when category popularity is predictive. Embedding layers: for very high cardinality in neural networks. Binary/hash encoding: compact representation for high-cardinality with tree models. Avoid label encoding for nominal features with linear models as it implies false ordinality.

8 Bias - Variance Tradeoff and Class Imbalance in Finance

Q How does the bias-variance tradeoff interact with class imbalance in a financial fraud detection setting?

A High class imbalance pushes models toward the majority class (low bias for majority, high bias for minority). A high-variance model overfits to the rare fraud patterns in training data but fails to generalize. Mitigation: (1) Resampling — SMOTE adds synthetic minority samples reducing bias; under-sampling reduces variance. (2) Cost-sensitive learning — penalize misclassifying fraud more heavily, shifting the decision boundary. (3) Use ensemble methods (Random Forest, XGBoost) that naturally reduce variance. (4) Evaluate on precision-recall curves, not accuracy or ROC-AUC. Calibrate probabilities post-training for reliable threshold setting.

9 Bias vs. Variance Tradeoff

Q Explain the bias-variance tradeoff and how it guides model selection.

A Bias: error from wrong assumptions in the model (underfitting). High-bias models (linear regression on non-linear data) have consistently wrong predictions. Variance: error from sensitivity to training data fluctuations (overfitting). High-variance models (deep decision trees) perform well on train but poorly on unseen data. Total error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$. Simple models have high bias/low variance; complex models have low bias/high variance. Solutions: reduce bias by adding features/complexity; reduce variance by regularization (L1/L2), more data, dropout, or pruning. Cross-validation helps estimate both.

10 Skewed Pricing

Q Your target variable (house prices) is heavily right-skewed. How does this affect your model, and what do you do?

A Skewed targets violate the normality assumption in linear regression, inflate MSE for high-value outliers, and make the model underestimate extremes. Solutions: (1) Log-transform the target ($\log_1 p$) — makes distribution more normal, predictions are then exponentiated back. (2) Use models robust to skewness: gradient boosted trees don't require normality. (3) Use MAE instead of MSE as loss (less sensitive to outliers). (4) Quantile regression if you want to predict specific percentiles. Always check residual plots post-training to verify the transform helped.

11 Softmax vs Logistic

Q What is the difference between softmax and logistic (sigmoid) regression, and when do you use each?

A Sigmoid maps a single score to $[0,1]$ — used for binary classification. Output is $P(y=1)$. Softmax generalizes to K classes: it takes K raw scores and outputs a probability distribution summing to 1: $P(y=k) = \exp(z_k) / \sum(\exp(z_j))$. Use sigmoid for binary problems; softmax for multi-class. With two classes, softmax reduces to sigmoid. Softmax assumes mutually exclusive classes; for multi-label problems (multiple classes can be true simultaneously), use independent sigmoid outputs per label. Numerically, subtract $\max(z)$ before exponentiation for stability.

12 Data Preparation for Imbalanced Data

Q What techniques would you use to prepare data for training a model on an imbalanced dataset?

A (1) Oversampling minority: SMOTE generates synthetic samples by interpolating between existing minority neighbors. (2) Undersampling majority: random undersampling or Tomek Links to remove borderline majority samples. (3) Combined: SMOTEENN or SMOTETomek. (4) Class weights: set `class_weight='balanced'` in sklearn — no data augmentation needed. (5) Threshold tuning: default 0.5 threshold is suboptimal; tune on validation set to maximize F1 or recall at acceptable precision. (6) Stratified splits: always use stratified train/val/test splits. Evaluate with PR-AUC and F1, not accuracy.

13 Top-K vs Nucleus (Top-P) Sampling in LMs

Q You are building a text generation system. Explain top-k and nucleus (top-p) sampling, their differences, and trade-offs.

A Top-k sampling: at each step, restrict sampling to the top k most probable tokens; sample from that fixed-size set. Risk: k may be too large (includes low-prob garbage) or too small (misses valid options) depending on distribution shape. Nucleus (top-p) sampling: sample from the smallest set of tokens whose cumulative probability exceeds p (e.g., 0.9). The set size adapts — small when distribution is peaked, larger when it's flat. This is generally preferred as it adapts to context. Trade-offs: top-k gives more control over diversity; top-p is more contextually appropriate. Combine both (top-k and top-p together) for best results. Lower p/k = more deterministic; higher = more creative.

14 Fine-Tuning VS RAG

Q You are building an internal chatbot at Thomson Reuters. How do you choose between fine-tuning an LLM vs. using Retrieval-Augmented Generation (RAG)?

A RAG: retrieves relevant documents at query time and injects them into the prompt. Best when: knowledge base updates frequently (news articles), content is too large for fine-tuning, you need citations/grounding, or you want to avoid hallucinations. Fine-tuning: trains the model on domain-specific data to internalize style, tone, or specialized reasoning. Best when: the task requires a specific output format, domain-specific reasoning patterns, or low-latency (no retrieval step). For a news-reference chatbot, RAG is preferred — articles change daily, grounding is critical, and retrieval keeps answers current. Hybrid: RAG for factual retrieval + fine-tuning for response style.

15 One Million Rides

Q You have data on 1 million ride-share trips. How would you predict ride duration?

A Features: pickup/dropoff coordinates, distance (haversine), hour of day, day of week, weather, surge pricing flag, traffic index (external API), historical average duration for that corridor. Model: LightGBM or XGBoost — handles non-linear interactions and is fast. Baseline: distance / average speed. Evaluate with MAE and RMSE; log-transform duration if skewed. Geospatial clustering (H3 hex bins) can encode location more efficiently than raw lat/long. For production: retrain daily, monitor MAE drift by time-of-day and geography.

16 Training Instability in Neural Networks

Q Your neural network loss diverges during training. What are the likely causes and how do you fix them?

A Causes and fixes: (1) Learning rate too high — use learning rate warmup, reduce LR, or use a scheduler (cosine annealing). (2) Exploding gradients — add gradient clipping (clip_grad_norm). (3) Vanishing gradients — use ReLU activations, batch normalization, residual connections. (4) Poor weight initialization — use He init for ReLU, Xavier for sigmoid/tanh. (5) Bad data — NaN/Inf values in inputs; add input validation and normalization. (6) Loss function mismatch — ensure correct pairing (cross-entropy for classification, MSE for regression). Monitor gradient norms, activation distributions (TensorBoard) to diagnose.

17 CNNs vs Intensity-Based Features

Q When would you use a CNN versus hand-crafted intensity-based features for an image classification task?

A Hand-crafted features (HOG, SIFT, LBP, pixel histograms): prefer when data is limited (<1000 images), interpretability matters, compute is constrained, or the signal is well-understood (e.g., detecting bright spots in microscopy). CNNs: prefer when you have large labeled datasets (>10k), the task is complex with subtle patterns, and you have GPU compute. CNNs learn hierarchical features automatically — edges → textures → objects. Transfer learning bridges the gap: use a pretrained CNN (ResNet, EfficientNet) fine-tuned on small datasets. If labeled data is scarce but unlabeled data is plentiful, consider self-supervised pretraining.

18 Transformer Encoder Layer

Q Design a Transformer encoder layer from scratch in PyTorch for processing customer support logs. Why are residual connections and layer normalization critical?

A Components: (1) Multi-head self-attention: Q, K, V projections; scaled dot-product attention; concatenate heads. (2) Add & Norm: residual connection + LayerNorm after attention. (3) Feed-forward network: two linear layers with ReLU/GELU. (4) Add & Norm again. Residual connections solve the vanishing gradient problem — gradients flow directly through skip connections during backprop, enabling very deep networks. Layer normalization stabilizes activations across the sequence length dimension (unlike BatchNorm which normalizes across the batch), making training stable regardless of sequence length or batch size. In PyTorch: use `nn.MultiheadAttention`, `nn.LayerNorm`, `nn.Linear`.

19 Keyword Bidding

Q You work at an ad-tech company. How would you build a model to optimize keyword bid prices in real-time auctions?

A Frame as a value-based bidding problem. Predict: (1) $P(\text{click} \mid \text{keyword}, \text{ad}, \text{user})$ — click-through rate model. (2) $P(\text{conversion} \mid \text{click})$ — conversion rate model. Expected value = $P(\text{click}) * P(\text{conv} \mid \text{click}) * \text{revenue_per_conversion}$. Set bid = EV * target_ROAS_fraction. Models: logistic regression (fast, interpretable) or LightGBM. Features: keyword, match type, quality score, historical CTR, time of day, device, user segment. Deploy as a low-latency scoring service (<10ms). Use Thompson Sampling or UCB for exploration of new keywords with sparse data. Monitor CPC, ROAS, and impression share.

20 Explaining PEFT

Q What is Parameter-Efficient Fine-Tuning (PEFT) and why choose it over full fine-tuning?

A PEFT methods fine-tune a small subset of parameters while keeping the pretrained LLM frozen. Key techniques: (1) LoRA (Low-Rank Adaptation): injects trainable low-rank matrices into attention layers — only 0.1–1% of parameters trained. (2) Prefix Tuning: prepends trainable tokens to each layer's input. (3) Adapter layers: small bottleneck modules inserted between transformer layers. Advantages over full fine-tuning: dramatically lower GPU memory and compute, faster training, avoids catastrophic forgetting of pretrained knowledge, easy to swap task-specific adapters without reloading the full model. Ideal when compute is limited or you need multiple task-specific model variants.

21 Mapping Nicknames

Q You have a database of user names and need to match them to a canonical list despite nicknames (Bob/Robert, Liz/Elizabeth). How do you approach this?

A Multi-strategy approach: (1) Exact match first. (2) Nickname dictionary lookup — maintain a curated nickname→canonical mapping (Bob→Robert). (3) Phonetic matching — Soundex or Metaphone for similar-sounding names. (4) Edit distance — Levenshtein distance for typos (threshold ≤ 2). (5) Embedding similarity — encode names with a character-level model and use cosine similarity. (6) Blocking — group by first letter or Soundex code to reduce comparison pairs before expensive matching. Combine signals with a logistic regression ranker trained on labeled match/no-match pairs. Evaluate with precision, recall, and F1 on held-out name pairs.

22 Netflix Churn Prediction

Q How would you build a churn prediction model for a subscription streaming service like Netflix?

A Define churn: cancellation within next 30 days. Features: recency (days since last watch), frequency (sessions/week), content diversity, watch-completion rate, payment history, plan type, device count, content preferences, support ticket history. Model: LightGBM or logistic regression; LSTM for sequential watch history. Handle class imbalance (churn ~3-8%): class weights or SMOTE. Evaluate: PR-AUC, F1. Calibrate probabilities for action thresholds. Output: churn probability score per user per week. Use scores to trigger retention interventions (discount offer, content recommendation). Measure lift via A/B test comparing intervention vs control.

23 Capital One Chatbot Design

Q Design a secure customer support chatbot for Capital One's online banking platform with access to account data, transactions, and FAQs.

A Architecture: RAG pipeline — embed FAQs and policy docs in a vector store (Pinecone/Weaviate); at query time retrieve top-k relevant chunks and inject into LLM prompt. For account-specific questions (balance, transactions), add a secure tool-calling layer: the LLM generates a structured API call, the backend validates user auth and returns data, the LLM formats the response. Security: never pass raw account data into the LLM prompt — only summarized/masked values. PII redaction before logging. Session-scoped auth tokens. Add guardrails: block prompt injection, restrict to banking topics, escalate to human agent for complaints/disputes. Evaluate with response accuracy, containment rate, CSAT.

24 Support Vector Machines vs Deep Learning Models

Q When would you choose an SVM over a deep learning model, and what are the trade-offs?

A SVMs work well when: data is small-to-medium (< 100k samples), features are well-engineered and high-dimensional (e.g., text with TF-IDF), a clear margin of separation exists, you need a convex optimization guarantee, or compute is limited. Deep learning works better when: data is large, raw features are unstructured (images, audio, text), hierarchical feature learning is beneficial, or transfer learning from pretrained models is available. SVMs have strong theoretical foundations, fewer hyperparameters, but don't scale well to millions of samples and require careful kernel selection. Deep learning is more flexible but needs more data, compute, and tuning.

25 A Simpler KNN From Scratch

Q Implement a simple K-Nearest Neighbors classifier from scratch and explain its key properties.
:

A Implementation: store all training points. For a query point, compute distances (Euclidean) to all training points, find the k nearest, return majority class label. Key properties: non-parametric (no training phase — lazy learner), decision boundary adapts to data density, time complexity $O(n \cdot d)$ per query (expensive at scale). k controls bias-variance: small k $\rightarrow$ low bias/high variance (overfitting); large k $\rightarrow$ high bias/low variance. Feature scaling is critical — KNN is distance-based. Improvements: KD-trees or ball trees for faster search; weighted voting by inverse distance; cross-validate k. Weakness: performs poorly in high dimensions (curse of dimensionality).

26 FAQ Matching

Q You have 500 FAQ entries. How would you build a system to match user queries to the most relevant FAQ?
:

A Semantic search approach: embed each FAQ question+answer using a sentence transformer model (e.g., sentence-transformers/all-MiniLM-L6-v2). Store embeddings in a vector index (FAISS). At query time, embed the user question and retrieve top-k nearest FAQ entries by cosine similarity. Re-rank with a cross-encoder model for precision. Fallback: BM25 keyword search for exact-match queries. Hybrid: combine BM25 score + embedding similarity (reciprocal rank fusion). Threshold: if best match score < threshold, route to human agent. Evaluate with MRR@k and NDCG on labeled query-FAQ pairs. Retrain embeddings periodically as FAQ content evolves.

27 Transformer Self-Attention

Q How does a transformer compute self-attention using Q, K, V, and why is masking needed in the decoder?
:

A Self-attention: given input X, compute $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Attention scores = $\text{softmax}(QK^T / \sqrt{d_k}) * V$. Each position attends to all others, weighted by similarity. Scaling by $\sqrt{d_k}$ prevents dot products from growing too large, keeping softmax gradients stable. Multi-head attention runs h parallel attention heads, each learning different relationship patterns, then concatenates and projects. Decoder masking (causal/autoregressive mask): during training, the decoder must not attend to future tokens (it would be cheating). An upper-triangular mask of -infinity zeroes out future positions after softmax, enforcing left-to-right generation.

28 Generative AI for Fraud Detection

Q As a consultant at Gartner, advise a fintech client on choosing between RAG, prompt engineering, or fine-tuning for fraud detection with 50,000 labeled transactions.
:

A Prompt engineering: zero/few-shot with transaction examples in the prompt. Fast to deploy, no training. Good for explanation generation, poor for high-volume real-time scoring. RAG: retrieve similar historical fraud cases and inject as context. Useful for explainability — 'this resembles case X.' Adds latency; retrieval quality matters. Fine-tuning: with 50k labeled examples, fine-tune a smaller LLM or a classifier head on embeddings. Best for pattern recognition and high-throughput scoring. Recommendation: use a fine-tuned classifier (or XGBoost) for real-time detection, and RAG-based LLM for post-hoc explanation generation to compliance teams. If dataset doubles to 100k, fine-tuning becomes even more compelling.

29 Feature Scaling and Normalization

Q When is feature scaling necessary, and what are the differences between standardization and min-max normalization?

A Scaling is necessary for: distance-based models (KNN, SVM, K-Means), gradient descent optimization (logistic/linear regression, neural networks), PCA. Not needed for: tree-based models (Decision Trees, Random Forest, XGBoost). Standardization (Z-score): $x' = (x - \text{mean}) / \text{std}$. Result has mean=0, std=1. Robust to outliers (relative to min-max). Preferred for normally distributed features and most ML algorithms. Min-max normalization: $x' = (x - \text{min}) / (\text{max} - \text{min})$. Scales to [0,1]. Sensitive to outliers. Useful for neural networks with bounded activation functions (sigmoid). RobustScaler: uses median and IQR — best when outliers are present.

30 Fraud Model Precision Drop

Q Your fraud detection model's precision dropped from 85% to 60% in production over 3 months. What would you investigate?

A Systematic investigation: (1) Data drift — compare feature distributions (PSI scores) between training data and recent production data; fraud patterns evolve. (2) Label drift — check if fraud labeling rules or dispute resolution process changed. (3) Class ratio shift — if fraud rate increased, more FPs at same threshold. (4) Feature pipeline issues — upstream data changes (new merchant categories, currency changes). (5) Threshold staleness — recalibrate decision threshold on recent data. (6) Adversarial adaptation — fraudsters learned to mimic legitimate patterns. Fix: retrain on recent data, adjust threshold, add new features capturing new fraud patterns, set up automated drift alerts.

31 85% vs 82%

Q Model A has 85% accuracy and Model B has 82% accuracy on the test set. Should you always prefer Model A?

A No. Accuracy is often misleading: (1) Class imbalance — with 95% negatives, a dummy model has 95% accuracy. (2) Different error costs — in fraud/medical diagnosis, false negatives are far more costly; compare F1, precision, recall, PR-AUC. (3) Calibration — Model B might have better-calibrated probabilities, making it more useful for decision-making. (4) Inference speed — Model B may be 10x faster. (5) Interpretability — simpler model B might be preferred in regulated industries. (6) Statistical significance — 3% difference may not be statistically significant on the test set. Always choose based on business-relevant metrics, not accuracy alone.

32 Video Game Respawn Model

Q In a video game, you want to predict the optimal respawn location for a player to minimize being killed immediately. How would you model this?

A Frame as a spatial risk minimization problem. Features per map location: historical enemy density (heatmap), time-since-last-death at each zone, player team positions, current kill streaks, game mode, time remaining. Model: predict $P(\text{killed within } T \text{ seconds} \mid \text{spawn at location } L)$ for each candidate spawn point — logistic regression or gradient boosted tree. At respawn time, score all candidate locations and pick the one with minimum predicted death probability. Training data: historical match logs with (spawn location, died_within_5s) labels. Add exploration: occasionally spawn in suboptimal locations to gather data. Validate by A/B testing survival rate against random/current respawn strategy.

33 Inherited Model Evaluation

Q You inherit an ML model from a colleague who left. How do you evaluate whether it's still performing well?
:

A (1) Understand the model: read documentation, check training data date, features used, target definition, and evaluation metrics. (2) Reproduce baseline: run the model on a held-out recent dataset; compare reported vs current metrics. (3) Check for data drift: plot feature distributions over time using PSI (Population Stability Index). (4) Check label drift: has the target definition or rate changed? (5) Review prediction distribution: has output score distribution shifted? (6) Business metrics: are downstream KPIs (revenue, churn prevented) still tracking? (7) Infrastructure: check for pipeline failures, stale feature sources. Document findings and set up automated monitoring before making changes.

34 Food Delivery Times

Q How would you build a model to predict food delivery time for a platform like DoorDash?
:

A Features: restaurant prep time (historical average + current queue), distance from restaurant to customer (routing API), traffic conditions, time of day/day of week, order complexity (item count), weather, courier availability score, payment processing time. Model: LightGBM for fast inference. Decompose the problem: predict prep time separately from transit time, sum for total ETA. Baseline: historical mean delivery time for restaurant. Evaluate with MAE — customers are sensitive to underestimates (promises broken) more than overestimates. Add a buffer (80th percentile estimate) to reduce complaints. Monitor per-restaurant and per-zone accuracy to detect outliers.

35 Ride Requests Model

Q Build a model to predict ride-share demand (number of ride requests) at a given location and time.
:

A Spatiotemporal forecasting. Aggregate requests to H3 hex grid cells at 15-minute intervals. Features: lag demand (t-1, t-4, t-96 = yesterday same time, t-672 = last week), hour, day of week, month, weather (rain increases demand), events (concerts, sports), holidays, distance to transit hubs. Model: LightGBM with lag features works well; alternatively a Seq2Seq LSTM or Temporal Fusion Transformer for multi-step forecasting. Evaluate with MAE/RMSE per cell. Use predictions to pre-position drivers (surge routing). Retrain daily. Monitor forecast error by zone — high-density zones need tighter accuracy.

36 Training vs Validation vs Test Data

Q Explain the difference between training, validation, and test sets and why all three are necessary.
:

A Training set: data the model learns from — parameters are optimized here. Validation set: held-out data used during development to tune hyperparameters and select the best model — not used in gradient updates. Test set: completely held-out data used exactly once to report final unbiased performance — simulates real-world deployment. Why all three: if you tune on test data, you inadvertently overfit to it (data leakage) and your reported metric is optimistically biased. Validation allows iterative improvement without contaminating the test set. Common splits: 70/15/15 or 80/10/10. For time-series, use temporal splits. In small datasets, use cross-validation to maximize data usage.

37 Optimize Model Performance

Q Your model is underperforming. Walk through a structured process to improve it.
:

A Systematic approach: (1) Diagnose — is it high bias (underfitting) or high variance (overfitting)? Compare train vs val error. (2) High bias fixes: add features, increase model complexity, reduce regularization, try a more powerful model. (3) High variance fixes: more training data, regularization (L1/L2/dropout), feature selection, ensemble methods, reduce model complexity. (4) Feature engineering: create interaction terms, domain-specific features, better encoding. (5) Hyperparameter tuning: random search then Bayesian optimization. (6) Data quality: fix label noise, outliers, leakage. (7) Ensembling: stack multiple models. (8) Calibration: calibrate probabilities (Platt scaling, isotonic regression). Document each experiment.

38 Batch vs Mini-Batch vs Stochastic Gradient Descent

Q Compare batch, mini-batch, and stochastic gradient descent. When would you use each?
:

A Batch GD: computes gradient on entire dataset per update. Stable, exact gradient, but very slow per update and requires all data in memory. Used rarely in practice. Stochastic GD (SGD): one sample per update. Fast, noisy gradient — can escape local minima but high variance. Mini-batch GD: n samples (typically 32-256) per update. Best of both: vectorized GPU efficiency, stable-enough gradient, can escape local minima. Used in nearly all deep learning. Batch size trade-offs: smaller batches → more noise (regularizing effect, better generalization); larger batches → faster convergence per epoch but may converge to sharp minima. Adjust learning rate when changing batch size (linear scaling rule).

39 Risk Assessment Model

Q You are building a credit risk assessment model at a bank. What considerations are most important?
:

A (1) Fairness/bias: ensure the model doesn't discriminate on protected attributes (race, gender) — use disparate impact analysis and fairness-aware training. (2) Regulatory compliance: models must be explainable (ECOA adverse action notices); use logistic regression or SHAP-explained GBMs. (3) Data: credit bureau features (FICO, DTI, payment history), application data, bank relationship data. Handle missingness carefully. (4) Evaluation: Gini coefficient, KS statistic, AUC-ROC on out-of-time validation (not random split). (5) Model stability: PSI monitoring, challenger model testing. (6) Reject inference: handle selection bias from only observing outcomes for approved applicants.

40 Seller Type Modeling

Q On a marketplace, how would you classify sellers as professional businesses vs. casual individuals?
:

A Behavioral features: listing frequency (items/week), price consistency, listing quality (description length, photo count), response time to messages, returns policy, product variety vs. specialization, account age, average order value, fulfillment speed. Model: LightGBM binary classifier (professional=1). Training labels: use known business accounts (VAT registered, verified business badge) as positives; infrequent low-volume sellers as negatives. Semi-supervised: use clustering on unlabeled sellers to find intermediate types. Evaluate with precision on high-confidence predictions. Output: probability score used to unlock seller features (bulk listing tools, analytics dashboard) or flag for human review.

41 Restaurant Recommender

Q Design a restaurant recommendation system for a food delivery app.
:

A Approaches: (1) Collaborative filtering: users with similar order history get similar recommendations (matrix factorization / ALS). (2) Content-based: match restaurant cuisine, price range, ratings to user preference profile. (3) Contextual: factor in time of day, weather, location, past re-order patterns. (4) Learning to Rank: train a ranker on implicit feedback (clicked, ordered, reordered) using LambdaMART or a two-tower neural model. Cold start: for new users, use location-based popularity; for new restaurants, use content features. Evaluate offline with NDCG, MRR; online with CTR and order conversion rate via A/B test. Re-rank for diversity to avoid filter bubble.

42 Explaining LDA

Q Explain Latent Dirichlet Allocation (LDA) and when you would use it.
:

A LDA is a generative probabilistic topic model. It assumes: each document is a mixture of topics; each topic is a distribution over words. Given a corpus, LDA infers: (1) topic-word distributions (what words characterize each topic) and (2) document-topic distributions (what topics each document covers). Inference via variational Bayes or Gibbs sampling. Hyperparameter alpha controls topic sparsity per document; beta controls word sparsity per topic. Use when: you want to discover latent themes in a large text corpus (news articles, reviews, support tickets) without labels. Alternatives: NMF (faster, deterministic), BERTopic (embedding-based, better coherence). Evaluate with topic coherence (C_v score) and perplexity.

43 Matching Siblings

Q You have a database of individuals and want to algorithmically identify pairs who are likely siblings. How would you approach this?
:

A Features for sibling likelihood: shared last name, shared address/household history, age gap within typical sibling range (0-20 years), shared emergency contacts, same school enrollment records, shared parental names in records. This is an entity resolution / record linkage problem. Step 1: blocking — only compare pairs with matching last name or address to reduce $O(n^2)$ comparisons. Step 2: feature engineering per pair — name similarity, age difference, address overlap duration. Step 3: train a binary classifier (sibling/not) on labeled pairs. Step 4: cluster connected components of high-confidence pairs. Evaluate with precision/recall on known sibling pairs. Handle privacy constraints carefully.

44 Financial Data Chatbot System

Q Design a chatbot system that provides real-time financial insights using frequently updated company data.
:

A Architecture: (1) Data ingestion: streaming pipeline (Kafka) ingests live financial data (prices, news, filings) → preprocessed and stored in a vector DB (updated continuously) and a structured DB (PostgreSQL/Redshift). (2) Retrieval: at query time, hybrid search (BM25 + vector similarity) retrieves relevant context. (3) LLM response generation: inject retrieved context + user query into LLM prompt with guardrails. (4) Tool use: for precise calculations (portfolio P&L, ratios), route to a deterministic calculation engine rather than asking the LLM to compute. (5) Caching: cache common queries (e.g., 'What is AAPL's current price?'). (6) Auth: responses scoped to user's data access level. Monitor latency, hallucination rate, user satisfaction.

45 Merchant Acquisition

Q You work at a payments company. How would you build a model to prioritize which merchants to target for acquisition?

A Frame as a lead scoring / ranking problem. Features: merchant industry (SIC code), estimated transaction volume (from web data, foot traffic APIs), current payment processor (from job postings, review sites), business age, Yelp/Google rating, social media presence, website traffic, proximity to existing merchant network. Target: $P(\text{merchant signs up within 90 days} \mid \text{contacted})$. Model: LightGBM ranker trained on historical outreach outcomes. Output: ranked lead list for sales team. Segment by merchant size (SMB vs enterprise — different sales motion). Evaluate with precision@k (top k leads converted). Continuously retrain as signed merchant data accumulates.

46 Rebalance Probabilities

Q You trained a classifier on a dataset where the positive class was oversampled 10x. How do you calibrate the output probabilities for production?

A When training with oversampled positives, the model's raw probability estimates are inflated for the positive class. Correction: if the true positive rate in production is p_{true} and training positive rate is p_{train} , apply Platt scaling or the prior correction formula: $p_{\text{corrected}} = p_{\text{raw}} / (p_{\text{raw}} + (1 - p_{\text{raw}}) * (p_{\text{train}} / p_{\text{true}}))$. Alternatively: retrain on the natural class distribution (just use class_weight parameter) — the cleanest solution. Or use isotonic regression / Platt scaling on a held-out calibration set with the natural class ratio. Always validate calibration with a calibration curve (reliability diagram) — a well-calibrated model's predicted probabilities match observed frequencies.

47 Identifying Good Investors

Q You work at a fintech platform. How would you build a model to identify high-quality investors?

A Define 'good investor': risk-adjusted returns (Sharpe ratio), low emotional trading, diversification, long-term holding behavior. Features: portfolio diversity (Herfindahl index), average holding period, trade frequency, win/loss ratio, drawdown behavior during market crashes, reaction to volatility (trading volume spike during downturns), account tenure, deposit/withdrawal patterns. Model: clustering first (K-Means / HDBSCAN) to discover investor archetypes (long-term passive, active trader, panic seller). Then build a quality score: regression predicting 12-month risk-adjusted return. Use score to personalize product recommendations (robo-advisor features, premium research). Validate against realized returns on held-out time period.

48 Dynamic Demand Pricing

Q How would you build a dynamic pricing model that adjusts prices based on demand?

A Demand modeling: estimate price elasticity — run historical regression of $\log(\text{demand})$ on $\log(\text{price})$ plus controls (time, competitor price, promotions). Elasticity $e = \% \text{ change in demand} / \% \text{ change in price}$. Pricing optimization: given elasticity, maximize revenue = price * demand(price). Optimal price = $\text{cost} / (1 + 1/e)$ for elastic goods. Reinforcement learning approach: treat pricing as a contextual bandit — state = (time, inventory, competitor price, demand signals); action = price; reward = revenue. Use Thompson Sampling or UCB. Constraints: price floors/ceilings, fairness (avoid discriminatory pricing). A/B test pricing strategies. Monitor conversion rate, revenue per session, and customer lifetime value impact.

49

Spanish Scrabble

Q Build an algorithm to find the highest-scoring valid word from a set of letter tiles in Spanish Scrabble.
:

A Algorithm: (1) Load a Spanish word dictionary (DRAE) as a set for $O(1)$ lookup. (2) Generate all permutations of subsets of available tiles (up to 7 tiles in Scrabble). (3) Filter: keep only permutations that exist in the dictionary. (4) Score each valid word: sum tile point values (note Spanish Scrabble has different tile distributions — LL, CH, RR in some editions). (5) Return highest-scoring word. Optimization: instead of generating all permutations, use a Trie of the dictionary — traverse valid prefixes to prune the search space dramatically. For blank tiles: treat as any letter. Time complexity with Trie: $O(7! * \text{branching factor})$ in worst case but pruned significantly in practice.

50

Evaluate News

Q How would you build a system to evaluate the quality and credibility of news articles at scale?
:

A Multi-signal approach: (1) Source credibility: domain reputation score (from fact-checking databases like NewsGuard), historical accuracy, editorial standards. (2) Content analysis: claim extraction using NLP → fact-checking against knowledge bases (Wikidata, verified sources). Claim consistency across multiple outlets. (3) Writing quality: readability score, sensationalism detection (clickbait classifier), emotional language ratio. (4) Author signals: author credibility score, publication history. (5) Social signals: sharing patterns, bot detection on amplifying accounts. Model: ensemble of the above signals into a credibility score (0–100). Use human fact-checker labels as training data. Evaluate with inter-annotator agreement. Surface score to readers as a browser extension or platform badge.